

Thẻ 1

NLP – Phân tích đánh giá, cảm xúc của khán giả sau khi xem phim

[Use Sentiment Analysis With Python to Classify Movie Reviews – Real Python](#)

[Sentiment Analysis on IMDB Movie Reviews - GeeksforGeeks](#)

Mục tiêu của đề tài *Sentiment Analysis on Movie Reviews* (Phân tích cảm xúc trong đánh giá phim) là xây dựng một mô hình học máy (Machine Learning) hoặc học sâu (Deep Learning) có khả năng tự động nhận diện cảm xúc (tích cực, tiêu cực hoặc trung lập) từ các đoạn bình luận phim do người dùng đăng tải trên các nền tảng trực tuyến như IMDb, Rotten Tomatoes hoặc Kaggle dataset.

Cụ thể, đề tài hướng đến:

- Xây dựng một pipeline xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP).
 - Huấn luyện và đánh giá mô hình phân loại cảm xúc (Sentiment Classification).
 - Tối ưu độ chính xác và khả năng tổng quát của mô hình trên dữ liệu thực tế.
- ⇒ Kết quả đạt được giúp mô hình **tự động nhận biết được xu hướng cảm xúc** trong đánh giá phim, hỗ trợ các hệ thống gợi ý phim, marketing, hoặc nghiên cứu hành vi người xem.

Dự án Phân tích cảm xúc đa lớp trên đánh giá phim IMDb

Giờ là bé N có bỏ cách trình bày luôn trên gg colab cho ae dễ follow nhé

Mục tiêu (TUI) (Phải xong trong tuần này)

- Phân loại đánh giá phim thành 3 lớp cảm xúc: Tích cực (Positive), Tiêu cực (Negative), Trung tính (Neutral).
- Xác định các tiêu chí đánh giá hiệu năng của mô hình (accuracy, precision, recall, F1).
- Đảm bảo dữ liệu cân bằng (25.000 mẫu cho mỗi lớp) với tổng cộng 75.000 mẫu.
- Khẳng định **spaCy TextCategorizer (CNN)** được sử dụng làm mô hình chính; các mô hình khác (Logistic Regression, BERT, DistilBERT, BiLSTM, Linear Regression) chỉ làm benchmark để so sánh.

PHẦN 0 — GIỚI THIỆU & MỤC TIÊU

Markdown:

- Giới thiệu bài toán phân tích cảm xúc là gì.
IMDb dataset: 2 lớp gốc (pos/neg).
- Mục tiêu mở rộng lên **3 lớp: Negative – Neutral – Positive**.
- Tổng quan mô hình:
 1. Mô hình chính: **spaCy TextCategorizer (CNN – spaCy 3.x)**.
 2. Benchmark: LR, Linear Regression (baseline), BiLSTM, BERT, DistilBERT.
- Pipeline tổng quát của project:
 1. Chuẩn bị dữ liệu
 2. EDA
 3. NLP preprocessing
 4. Huấn luyện
 5. Đánh giá
 6. Web app demo

(Không chạy code ở phần này.)

Chuẩn bị dữ liệu (ĐÚC) (Tối thứ 2 / hợp)

- Thu thập dữ liệu gốc từ IMDb (gồm các đánh giá tích cực và tiêu cực có sẵn, bổ sung dữ liệu Trung tính từ nguồn khác hoặc tự tạo).
- Mở rộng tập dữ liệu lên 75.000 mẫu bằng cách thu thập thêm đánh giá hoặc tạo nhãn Trung tính phù hợp.
- Làm sạch dữ liệu: loại bỏ HTML, ký tự đặc biệt; chuẩn hóa văn bản (chuyển chữ hoa thành chữ thường, tiền xử lý ngôn ngữ).
- Gắn nhãn dữ liệu rõ ràng theo 3 lớp cảm xúc (Tích cực = 0, Tiêu cực = 2, Trung tính = 1).
- Phân chia tập dữ liệu thành tập huấn luyện, tập validation và tập test (ví dụ: 70% huấn luyện = 52500, 15% validation = 11250, 15% test = 11250).
- (--- QUY TRÌNH LÀM ---)
- - Tải file data neutral(trung lập) từ link: [Dataset-Neutral](#)
- - Tìm cách xử lý file data neutral từ tiếng Nga sang tiếng Anh:

PHẦN 1 — CHUẨN BỊ DỮ LIỆU

Markdown:

Giải thích lý thuyết:

- Nguồn dữ liệu IMDb (pos/neg).
- Dataset neutral (ban đầu là tiếng Nga → dịch sang English).
- Lý do phải làm sạch dữ liệu và chuẩn hóa để mô hình học đúng.
- Chuẩn hoá nhãn:
 - Negative = 0
 - Neutral = 1

- Positive = 2

Code:

- Load raw IMDb data.
 - Load neutral dataset.
Dịch neutral sang tiếng Anh (hoặc load data đã dịch).
 - Gộp vào 1 DataFrame.
 - Loại HTML, emoji, ký tự đặc biệt.
 - Lowercase và chuẩn hóa khoảng trắng.
 - Encode nhãn.
- Output:**
- Hiển thị 5 dòng đầu.
 - Hiển thị tổng số mẫu mỗi lớp.

Khám phá dữ liệu (EDA) (DUY) (Phải xong trong tuần này)

- Phân tích phân bố số lượng mẫu của từng lớp cảm xúc (Positive, Negative, Neutral) để xác định tính cân bằng dữ liệu.
- Xác định các từ khóa phổ biến trong mỗi lớp (ví dụ: phân tích tần suất từ, n-gram). (Chưa có)
- Tạo biểu đồ và trực quan hóa dữ liệu: word cloud, biểu đồ thanh, biểu đồ tròn thể hiện tỷ lệ từng lớp và từ vựng đặc trưng. (Chưa có)
- Kiểm tra chất lượng dữ liệu: loại bỏ dữ liệu trùng lặp, dữ liệu không hợp lệ hoặc chứa lỗi rõ ràng. (Cũng trùng 1 nửa)
- Nếu phát hiện mất cân bằng dữ liệu nào, thực hiện các biện pháp điều chỉnh (nhân bản mẫu, thu thập thêm dữ liệu...).

PHẦN 2 — EDA (Exploratory Data Analysis)

Markdown:

Giải thích:

- EDA giúp hiểu dữ liệu, kiểm tra cân bằng, phát hiện lỗi.

Các thống kê cần xem:

- Class distribution
- Length distribution
- WordCloud
n-grams
- Duplicate checking

Code:

- Vẽ bar chart phân bố nhãn.
- Tính độ dài review + vẽ histogram.
- Loại duplicate + đếm lại.
- WordCloud cho pos/neg/neu.
- Top bigrams/trigrams.

Output:

- 3 WordCloud
- 2 biểu đồ phân bố
- Hình thống kê độ dài review
- Nhận xét ngắn cuối phần.

Xử lý ngôn ngữ tự nhiên (NLP) (PHÁT) (Tối thứ 2 / hợp)

- Tiền xử lý văn bản: tokenization, chuyển văn bản về dạng chuẩn (lowercasing, loại bỏ dấu câu), lemmatization hoặc stemming nếu cần.
- Loại bỏ stop words, ký tự số, biểu tượng cảm xúc (emojis) và các ký tự đặc biệt còn lại. (Chưa có)

- Biểu diễn văn bản dưới dạng số học: vector hóa văn bản bằng spaCy (word vectors), TF-IDF hoặc embeddings (như Word2Vec, GloVe) tùy theo yêu cầu của từng mô hình. (Chưa có)
- Cài đặt và cấu hình pipeline spaCy với thành phần **TextCategorizer** để thực hiện phân loại cảm xúc. (Chưa có)
- (Tùy chọn) Sử dụng embedding từ các mô hình pre-trained (như GloVe, BERT) nếu triển khai BiLSTM hoặc các biến thể khác. (Chưa có)

PHẦN 3 — NLP PREPROCESSING & VECTOR HOÁ

Markdown:

- Giải thích tokenization, lemmatization, stopwords.
- Vì sao spaCy xử lý tốt cho CNN.
- Vectorization từng mô hình:
 - spaCy CNN dùng embedding của spaCy.
 - Logistic Regression dùng TF-IDF.
 - BiLSTM dùng embedding (GloVe).
 - BERT/DistilBERT dùng transformer embedding. (*)

Code:

- spaCy tokenize demo.
- TF-IDF example.
- Load GloVe (nếu dùng).
- Demo tokenization/lemmatization.

Huấn luyện mô hình (TUI & ĐÚC & DUY) (Tối thứ 2 / học)

- **spaCy TextCategorizer (CNN):** Xây dựng và huấn luyện mô hình phân loại với pipeline spaCy.

- Đặt và điều chỉnh siêu tham số: số epoch, learning rate, dropout, batch size.
- Huấn luyện mô hình trên tập huấn luyện và đánh giá trên tập validation để theo dõi hiệu suất.
- Lưu trữ mô hình đã huấn luyện cùng các thông số cấu hình để sử dụng sau này.
- **Mô hình benchmark:** Huấn luyện các mô hình đối chứng để so sánh hiệu quả: (Đức Duy) (20/12)
 - Logistic Regression: sử dụng TF-IDF hoặc CountVectorizer để huấn luyện và đánh giá.
 - BERT và DistilBERT: Fine-tuning trên tập dữ liệu mở rộng (sử dụng thư viện Transformers).
 - BiLSTM: Xây dựng mạng neural hai chiều với embedding, huấn luyện và đánh giá hiệu năng.
 - Linear Regression: (dù thường dùng cho hồi quy, chỉ dùng ở đây để tham khảo và so sánh).
- Đánh giá hiệu năng của từng mô hình bằng các chỉ số: accuracy, precision, recall, F1 trên tập test.
- Vẽ và phân tích ma trận nhầm lẫn (confusion matrix) cho từng mô hình để hiểu rõ các lỗi phân loại. → Tham khảo thêm
- Ghi lại kết quả huấn luyện, thời gian huấn luyện và các tham số của mỗi mô hình để so sánh.

PHẦN 4 — HUẤN LUYỆN MÔ HÌNH

4.1 — spaCy TextCategorizer (CNN — Model chính)

Markdown:

- Kiến trúc CNN trong textcat (spaCy).
- Siêu tham số: lr, batch size, epochs, dropout.

- Cách định dạng dữ liệu theo spaCy (`{"cats": ...}`).

Code:

- Convert train/val/test sang dạng spaCy.
- Tạo pipeline spaCy.
- Train theo epochs.
- Lưu model `spacy_cnn_multiclass`.

4.2 — Benchmark Models

Markdown (mỗi mô hình 1 phần nhỏ):

- Logistic Regression (TF-IDF).
Linear Regression (nhấn mạnh: baseline không phù hợp).
- BiLSTM (kiến trúc + embedding).
- BERT / DistilBERT (fine-tune 3 lớp).

Code:

- Train LR + evaluate.
- Train Linear Regression baseline.
- Train BiLSTM.
- Fine-tune DistilBERT.

Phân loại đánh giá mới (TUI)

- Xây dựng chức năng hoặc API để nhập một đánh giá phim mới và phân loại nó.
- Sử dụng mô hình spaCy chính (TextCategorizer) để dự đoán nhãn cảm xúc của đánh giá mới.
- Đánh giá độ chính xác của phân loại trên các ví dụ mới (có nhãn thật) để kiểm thử tính hiệu quả.

- Thực hiện thử nghiệm với các đánh giá thực tế để minh họa hiệu quả của mô hình trong thực tế.

PHẦN 5 — ĐÁNH GIÁ MÔ HÌNH

Markdown:

- Giải thích metrics: Accuracy / Precision / Recall / F1 / Macro.
- Nêu lý do dùng F1 macro (dữ liệu 3 lớp).

Code:

- Evaluate từng model.
- Vẽ confusion matrix cho spaCy CNN và 1–2 model khác.
- Tạo bảng so sánh tất cả mô hình.

Output:

- Bảng tổng hợp:
 - Accuracy
 - Precision_macro
 - Recall_macro
 - F1_macro
 - Training time

Markdown (nhận xét):

- spaCy CNN trade-off tốt (nhẹ, nhanh, accuracy hợp lý).
- BERT accuracy cao nhất nhưng nặng.
- LR nhanh nhưng yếu với dữ liệu dài.

Trực quan hóa và Web App (PHÁT) (20/12)

- Trực quan hóa kết quả huấn luyện: vẽ biểu đồ accuracy và loss qua các epoch để quan sát quá trình hội tụ.
- Trực quan hóa so sánh hiệu năng giữa các mô hình (ví dụ: biểu đồ cột cho accuracy, F1).
- Tạo giao diện web đơn giản (sử dụng Flask hoặc Streamlit) cho phép người dùng nhập review và xem kết quả phân tích.
- Hiển thị kết quả phân loại (nhãn cảm xúc và xác suất dự đoán) trên giao diện web.
- (Tùy chọn) Trực quan hóa đặc trưng quan trọng: hiển thị các từ khóa đóng góp lớn cho quyết định của mô hình (sử dụng công cụ explainability của spaCy nếu cần).

PHẦN 6 — WEB APP DEMO (Streamlit)

Markdown:

- Giải thích chức năng web app:
 - Nhập review → Chọn mô hình → Xuất nhãn + xác suất
 - Dashboard nhỏ:
 - Class distribution
 - Confusion matrix
 - Wordcloud
- Lý do không dùng Movie Explorer: dataset IMDb không chứa movie title.

Code:

- Code chạy Streamlit app (đơn giản).
- Load mô hình spaCy CNN.
- Dự đoán & trả về kết quả.

Output:

- Link/ảnh UI (nếu có).

- Hướng dẫn chạy:

```
streamlit run app.py
```

Kết luận và hướng phát triển (TUI)

- Tổng kết hiệu năng của **spaCy TextCategorizer (CNN)** và các mô hình benchmark trên tập dữ liệu đã sử dụng.
- Nhận xét ưu nhược điểm của spaCy CNN trong bài toán phân tích cảm xúc đa lớp (ví dụ: ưu thế về tốc độ, nhược điểm về khả năng xử lý ngữ cảnh...).
- Đề xuất cải tiến: mở rộng thêm dữ liệu, tinh chỉnh siêu tham số, thử nghiệm với các mô hình hiện đại hơn (như transformer mới hoặc mô hình ensemble).
- Định hướng phát triển: mở rộng hỗ trợ đa ngôn ngữ, cải thiện giao diện người dùng, tích hợp hệ thống vào ứng dụng thực tế.

PHẦN 7 — KẾT LUẬN & HƯỚNG PHÁT TRIỂN

Markdown:

- Tổng kết hiệu năng mô hình.
- spaCy CNN = lựa chọn deploy do cân bằng giữa tốc độ và hiệu suất.
- Hạn chế:
 - Data neutral dịch tự động → có noise.
 - Không xử lý sarcasm/irony.
- Hướng phát triển:
 - Augmentation cho neutral.
 - Fine-tune transformer hiện đại hơn.

- Deploy API + UI đầy đủ.
- Hỗ trợ đa ngôn ngữ (EN + VN...).

(Không cần code)

So sánh mô hình (ĐÚC VS DUY)

- Lập bảng so sánh kết quả (accuracy, precision, recall, F1, thời gian huấn luyện) của các mô hình: spaCy CNN, Logistic Regression, BERT, DistilBERT, BiLSTM, Linear Regression.
- **Phân tích sự khác biệt về hiệu năng và hiệu suất:** ví dụ spaCy CNN có tốc độ huấn luyện nhanh, dễ tích hợp; BERT và DistilBERT có độ chính xác cao nhưng đòi hỏi tài nguyên lớn; Logistic Regression và Linear Regression đơn giản nhưng hiệu quả giới hạn.
- **Kết luận về việc lựa chọn mô hình chính:** ưu tiên spaCy TextCategorizer do sự cân bằng giữa hiệu quả và độ phức tạp, trong khi các mô hình khác chỉ dùng để so sánh và đánh giá mức độ cải thiện.

ÉNG Lịch

FINAL ACADEMIC OUTLINE FOR THE NOTEBOOK

0. Introduction & Project Objectives

Markdown (theory)

- Introduce the problem of **Sentiment Analysis**, its relevance, and typical applications.
- Present the IMDb dataset: originally containing two sentiment classes (**positive/negative**).
- State the goal of extending the task to **multiclass sentiment classification**:
 1. **Negative (0)**
 2. **Neutral (1)**
 3. **Positive (2)**
- Highlight the project's main goals:
 1. Build a full NLP pipeline
 2. Train a multiclass model
 3. Compare performance across several models
 4. Deploy a lightweight Web App for demonstration
- Overview of models used:
 1. **Primary Model:** spaCy TextCategorizer (CNN, spaCy 3.x)
 2. **Benchmark Models:** Logistic Regression, Linear Regression (baseline), BiLSTM, BERT, DistilBERT
- High-level pipeline of the project:
 1. Data Preparation
 2. EDA

3. NLP Preprocessing
4. Model Training
5. Model Evaluation
6. Web App Demonstration

(No code in this section.)

1. Data Preparation

Markdown (theory)

Explain the data sources:

- IMDb dataset for **positive/negative** samples.
- A separate dataset containing **neutral** reviews originally in Russian, translated into English.
- Rationale for translation and integration.

Describe preprocessing goals:

- Remove HTML tags, URLs, emojis, and special characters.
- Normalize text (lowercasing, whitespace normalization).
- Encode labels numerically: 0 (neg), 1 (neu), 2 (pos).

Explain dataset splitting strategy:

- 70% training
- 15% validation
- 15% test

Code

- Load positive, negative, and neutral datasets.
- Apply translation (or load pre-translated neutral reviews).
- Clean text and merge datasets.
- Encode labels, shuffle, and split.

Output

- Display the first few rows.
- Display class counts.

2. Exploratory Data Analysis (EDA)

Markdown (theory)

Explain the purpose of EDA:

- Assess class distribution.
- Examine review length variation.
- Detect noise, anomalies, and duplicates.

- Identify vocabulary characteristics through n-grams and word clouds.

Code

- Plot class distribution (bar chart).
- Compute review lengths and plot histogram/boxplot.
- Detect and drop duplicate reviews.
- Generate word clouds for positive, neutral, and negative classes.
- Extract and display top n-grams per class.

Output

- Word clouds (3 classes).
- Length distribution plots.
- Cleaned statistics.
- Short conclusions summarizing EDA findings.

3. NLP Preprocessing & Text Vectorization

Markdown (theory)

Explain fundamental NLP steps:

- **Tokenization**
- **Lemmatization**

- **Stopword removal** (optional depending on model)

Describe vectorization strategies for each model:

- **spaCy CNN:** uses spaCy's internal embedding representation
- **Logistic Regression / Linear Regression:** TF-IDF or CountVectorizer
- **BiLSTM:** pretrained embeddings such as GloVe
- **BERT / DistilBERT:** transformer-based tokenization + contextual embeddings

Code

- Demonstrate tokenization and lemmatization using spaCy.
- Example TF-IDF vectorization.
- Load GloVe vectors if used.
- Tokenization examples for BERT.

Output

- Tokenization samples
- Embedded vector examples
- Short remarks

4. Model Training

4.1 Primary Model: spaCy TextCategorizer (CNN)

Markdown (theory)

Explain spaCy's text classification architecture:

- How CNN-based `textcat` works
- Input format required by spaCy (`{"cats": {"neg": 1/0, "neu": 1/0, "pos": 1/0}}`)
- Choice of hyperparameters:
 - learning rate
 - dropout
 - number of epochs
 - batch size

Code

- Convert dataset into spaCy-compatible format
- Build spaCy pipeline
- Train the CNN classifier
- Save the trained model to disk

4.2 Benchmark Models

Markdown (theory)

Provide a brief academic description for each:

- **Logistic Regression (TF-IDF baseline)**
 - Efficient linear classifier
 - Performs reasonably well with sparse vectors
- **Linear Regression (non-standard baseline)**
 - Not intended for classification
 - Included to demonstrate poor suitability and create a meaningful comparison baseline
- **BiLSTM**
 - Captures bidirectional contextual information
 - Requires word embeddings (trained or pretrained)
- **BERT / DistilBERT**
 - Transformer-based fine-tuning
 - Best performance but computationally heavy

Code

- Train LR + evaluate
- Train Linear Regression baseline
- Train BiLSTM

- Fine-tune BERT/DistilBERT for multiclass classification
-

5. Model Evaluation

Markdown (theory)

Explain evaluation metrics:

- **Accuracy**
- **Precision / Recall / F1-score (macro)** — suitable for multiclass
- Why F1-macro is the most reliable in imbalanced or near-balanced multiclass tasks

Code

- Predict on test set for each model
- Compute metrics
- Plot confusion matrix for:
 - spaCy CNN
 - and optionally BERT or Logistic Regression
- Create a comparison table of all models

Output

A table summarizing:

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	Training Time
-------	----------	----------------------	-------------------	---------------	------------------

Markdown (conclusion for the section)

- Discuss model behaviors
 - spaCy CNN: strong trade-off between speed and performance
 - BERT: highest accuracy but costly
 - LR: fast but limited
 - Linear Regression: inappropriate for classification (as expected)
-

6. Web App Demonstration (Streamlit)

Markdown (theory)

Explain:

- Purpose of the Web App: an interactive tool to demonstrate the model
- Features included:
 - Input field for user review
 - Dropdown menu to select the model
 - Prediction output (label + probability distribution)
 - Visualization components (word clouds, class distribution, confusion matrix)

- Justification for excluding a “Movie Explorer”:
 - IMDb datasets do not include movie titles paired with each review
 - The project focuses on review-level sentiment, not movie-level summarization

Code

- Load spaCy model
- Build Streamlit interface
- Implement prediction logic
- Display probability bar chart
- Display EDA visuals within the app

Output

- Screenshots (optional)

Instructions to run:

```
streamlit run app.py
```

-
-

7. Conclusion and Future Work

Markdown (theory)

Summarize:

- Performance comparison across models
- Why spaCy CNN is chosen as the deployment model
- Observations from EDA and model behavior

Discuss limitations:

- Neutral dataset created through machine translation — potential noise
- Difficulty detecting sarcasm, irony, and long-context sentiment
- BERT's computational cost

Propose future work:

- Data augmentation (especially for neutral class)
- Fine-tuning modern transformer models (RoBERTa, DeBERTa)
- Deploying a full backend API + improved UI
- Extending to multilingual sentiment analysis (English + Vietnamese)

(No code.)

Chuẩn bị dữ liệu-Đức

Chuẩn bị dữ liệu (ĐÚC)

- Thu thập dữ liệu gốc từ IMDb (gồm các đánh giá tích cực và tiêu cực có sẵn, bổ sung dữ liệu Trung tính từ nguồn khác hoặc tự tạo). **Done**
- Mở rộng tập dữ liệu lên 75.000 mẫu bằng cách thu thập thêm đánh giá hoặc tạo nhân Trung tính phù hợp. **Done**
- Làm sạch dữ liệu: loại bỏ HTML, ký tự đặc biệt; chuẩn hóa văn bản (chuyển chữ hoa thành chữ thường, loại bỏ stopwords, tiền xử lý ngôn ngữ). **Done**
- Gán nhãn dữ liệu rõ ràng theo 3 lớp cảm xúc (Tích cực = 0, Tiêu cực = 2, Trung tính = 1). **Done**
- Phân chia tập dữ liệu thành tập huấn luyện, tập validation và tập test (ví dụ: 70% huấn luyện = 52500, 15% validation = 11250, 15% test = 11250).
- (--- QUY TRÌNH LÀM ---)

1. **Tải file data neutral(trung lập) từ link: [Dataset-Neutral](#)**

2. **Xử lý dữ liệu IMDB gốc (positive/negative)**

→ File: **Tran8.py**

Mục đích:

- Chuyển dữ liệu từ **IMDB Dataset.csv** gốc (có cột review và sentiment) thành 2 cột chính: **text và label**.
- Gán nhãn số cho sentiment:
 - Positive → 0
 - Negative → 2
- **Làm sạch dữ liệu:**
 - Loại bỏ HTML tags và URLs
 - Loại bỏ ký tự đặc biệt, chỉ giữ chữ cái và dấu câu cơ bản
 - Chuyển về chữ thường
 - Loại bỏ stopwords tiếng Anh phổ biến
 - Loại bỏ các dòng text rỗng
- Kết quả lưu thành file **pos_neg_data.csv**.

3. **Xử lý dữ liệu trung tính từ tiếng Nga (neutral)**

→ Files: **Phat_code.py, Phat_code_2.py, Phat_code_3.py, Tran6.py**

Quy trình:

1. Dịch từ tiếng Nga sang tiếng Anh
 - Các file review trung tính ban đầu ở tiếng Nga, được lưu trong thư mục **neu_**.
 - Sử dụng **GoogleTranslator(gọi API để dịch)** với đa luồng để dịch từng file (*.txt) sang tiếng Anh.
 - Dịch từng đoạn nhỏ (~1000 ký tự) để tránh quá tải request và tăng tốc độ.
 - Kết quả dịch được lưu trong thư mục neu_1/.
2. Chuyển sang **1 dòng và chuẩn hóa** để đưa vào CSV
 - Các file dịch xong được ghép thành 1 dòng và thêm **_neu** ở cuối.
 - Mỗi dòng tương ứng với 1 review trung tính.
 - Kết quả này có thể trực tiếp nhập vào CSV với nhãn 1 (Neutral).

4. Gộp dữ liệu và xáo trộn (final dataset)

→ File: **Tran9.py**

Quy trình:

1. Đọc 2 file CSV đã chuẩn hóa:
 - **pos_neg_data.csv** → chứa review tích cực và tiêu cực
 - **neutral_data.csv** → chứa review lập đã dịch từ Nga sang Anh
2. Gộp dữ liệu:
 - Sử dụng **pd.concat** để hợp nhất thành 1 DataFrame. (**Tran9.py**)
 - Xáo trộn toàn bộ dữ liệu để tránh thứ tự bias.
3. Lưu kết quả cuối cùng:
 - File hợp nhất: **final_combined_data.csv**
 - Các cột:
 - text: review đã làm sạch
 - label: 0, 1, 2
 - Có thống kê số lượng mẫu theo từng nhãn.

```
Phân bố nhãn trong TỔNG HỢP (sau khi gộp):
-----
Label 0 - Tích cực (Positive)      : 25000 mẫu (33.34%)
Label 1 - Trung tính (Neutral)     : 24991 mẫu (33.33%)
Label 2 - Tiêu cực (Negative)      : 25000 mẫu (33.34%)
TỔNG CỘNG                          : 74991 mẫu
-----

KIỂM TRA CHẤT LƯỢNG:
-----
• Số ô trống (null): 0
• Số dòng text rỗng: 0
• Label không hợp lệ: 0
-----
```


0. Gioi thieu va Muc tieu

0. Giới thiệu và Mục tiêu của Dự án

Phân tích cảm xúc (Sentiment Analysis) là một nhiệm vụ quan trọng trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing – NLP), tập trung vào việc nhận diện và diễn giải sắc thái cảm xúc xuất hiện trong văn bản. Bằng cách lượng hóa thông tin chủ quan, phân tích cảm xúc giúp các hệ thống và tổ chức hiểu rõ hơn thái độ và hành vi của người dùng.

Bộ dữ liệu đánh giá phim IMDb từ lâu đã được xem như một bộ dữ liệu chuẩn trong nghiên cứu phân loại cảm xúc. Trong dạng nguyên gốc, dataset này chỉ bao gồm hai nhãn cảm xúc — **tích cực (positive)** và **tiêu cực (negative)** — phù hợp cho bài toán phân loại nhị phân. Tuy nhiên, cảm xúc của người dùng trong thực tế thường mang tính đa dạng và không phải lúc nào cũng cực đoan theo hai chiều tích cực hay tiêu cực. Để phản ánh tốt hơn tính phức tạp đó, dự án này mở rộng nhiệm vụ phân loại nhị phân thành **phân loại cảm xúc đa lớp (multiclass)** với ba mức độ:

- **Tiêu cực (0)**
- **Trung tính (1)**
- **Tích cực (2)**

Cách tiếp cận này cho phép mô hình nắm bắt sắc thái cảm xúc tinh vi hơn và mang tính ứng dụng thực tiễn cao hơn.

Dự án hướng đến các mục tiêu chính sau:

1. **Xây dựng một pipeline NLP hoàn chỉnh**, bao gồm tiền xử lý dữ liệu, biểu diễn văn bản, huấn luyện mô hình và triển khai mô hình.
2. **Huấn luyện mô hình phân loại cảm xúc đa lớp**, sử dụng kiến trúc học sâu hiện đại.
3. **Thực hiện so sánh nhiều mô hình học máy và học sâu**, nhằm đánh giá hiệu năng giữa các phương pháp NLP khác nhau.
4. **Triển khai một ứng dụng Web đơn giản**, cho phép người dùng nhập đánh giá phim và nhận kết quả phân loại theo thời gian thực.

Trong số các mô hình được xem xét, dự án lựa chọn **spaCy TextCategorizer (kiến trúc CNN, spaCy 3.x)** làm mô hình chính nhờ khả năng huấn luyện nhanh và dễ dàng triển khai. Các mô hình khác được dùng làm hệ thống đối chứng, bao gồm:

- **Logistic Regression**
- **Linear Regression** (đóng vai trò baseline, dù không phù hợp cho phân loại)
- **BiLSTM**
- **BERT**
- **DistilBERT**

Quy trình tổng thể của dự án được thiết kế theo một pipeline chặt chẽ và rõ ràng:

1. **Chuẩn bị và làm sạch dữ liệu**
2. **Phân tích dữ liệu khám phá (EDA)**
3. **Tiền xử lý NLP và vector hóa văn bản**
4. **Huấn luyện mô hình**
5. **Đánh giá và so sánh mô hình**
6. **Trình diễn mô hình thông qua Web App**

Pipeline này đảm bảo tính hệ thống, dễ tái lập và mang lại cái nhìn toàn diện về quy trình xây dựng một mô hình phân tích cảm xúc đa lớp trong môi trường ứng dụng thực tế.

4. Huan luyen mo hinh

4. Model Training

4.1. spaCy TextCategorizer (CNN — Mô hình chính)

4.1.1. Kiến trúc CNN trong spaCy TextCategorizer

Trong bài toán phân loại cảm xúc đa lớp, mô hình chính được sử dụng là **spaCy TextCategorizer** với kiến trúc **Convolutional Neural Network (CNN)**.

Ý tưởng tổng quát:

- Mỗi review văn bản ban đầu được token hóa và chuyển thành chuỗi vector embedding:
 - Ký hiệu:
 - văn bản đầu vào: x
 - chuỗi embedding: $E(x) = [e_1, e_2, \dots, e_T]$
trong đó T là độ dài câu, e_i là vector đặc trưng của từ thứ i .
- Các **bộ lọc tích chập (convolution filters)** có kích thước khác nhau (ví dụ 3, 4, 5) được trượt trên chuỗi embedding để trích xuất đặc trưng cục bộ.
 - Với mỗi kích thước kernel k , ta có một bộ lọc w_k .
 - Tại mỗi vị trí cửa sổ, ta tính một giá trị đặc trưng (sau phi tuyến, ví dụ ReLU).
 - Chuỗi giá trị này mô tả “mức độ xuất hiện” của một mẫu ngữ nghĩa nào đó trong câu.
- Sau khi áp dụng convolution, spaCy thực hiện **max-pooling** trên toàn bộ trục thời gian (theo chiều dài câu) để giữ lại giá trị đại diện mạnh nhất của mỗi filter.
 - Kết quả, với mỗi bộ lọc ta thu được một số thực.
 - Nối tất cả các số thực từ nhiều filter, ta có một vector đặc trưng cuối cùng cho câu, gọi là $h_{cnn}(x)$.
- Vector $h_{cnn}(x)$ được đưa vào **lớp phân loại tuyến tính** và sau đó qua **softmax** để thu được phân bố xác suất trên ba lớp cảm xúc:
 - $z(x) = W * h_{cnn}(x) + b$

- $y_{\text{hat}}(x) = \text{softmax}(z(x))$

Trong đó:

- W và b là các tham số của lớp phân loại cuối cùng.
- $y_{\text{hat}}(x)$ là vector gồm 3 phần tử:
 - $y_{\text{hat_neg}}(x)$ — xác suất là Negative
 - $y_{\text{hat_neu}}(x)$ — xác suất là Neutral
 - $y_{\text{hat_pos}}(x)$ — xác suất là Positive

Mô hình CNN trong spaCy được hiện thực nội bộ (không cần tự viết lại), nhưng về bản chất nó vẫn tuân theo cấu trúc như trên.

4.1.2. Hàm mất mát và mục tiêu tối ưu

Vì đây là bài toán **phân loại đa lớp**, mô hình được huấn luyện bằng **hàm mất mát cross-entropy đa lớp**.

Ký hiệu:

- Có N mẫu trong tập huấn luyện.
- Với mỗi mẫu thứ i , nhãn thật được biểu diễn theo dạng one-hot:
 - $y_{\{i, \text{neg}\}}, y_{\{i, \text{neu}\}}, y_{\{i, \text{pos}\}} \in \{0, 1\}$
- Dự đoán của mô hình là:
 - $y_{\text{hat}_{\{i, \text{neg}\}}}, y_{\text{hat}_{\{i, \text{neu}\}}}, y_{\text{hat}_{\{i, \text{pos}\}}} \in [0, 1]$
 - và $y_{\text{hat}_{\{i, \text{neg}\}}} + y_{\text{hat}_{\{i, \text{neu}\}}} + y_{\text{hat}_{\{i, \text{pos}\}}} = 1$.

Hàm mất mát được định nghĩa:

- $L(\theta) = -(1/N) * \sum_i \sum_c y_{\{i, c\}} * \log(y_{\text{hat}_{\{i, c\}}})$

Trong đó:

- **theta** là toàn bộ tham số mô hình (embedding, trọng số các lớp CNN, W, b).
- Chỉ số **c** chạy qua 3 lớp: neg, neu, pos.

Mục tiêu huấn luyện:

- Tìm bộ tham số **theta*** sao cho **L(theta*)** nhỏ nhất trên tập huấn luyện.
- spaCy sử dụng các biến thể của gradient descent (Adam, SGD) để cập nhật **theta** theo hướng giảm dần hàm mất mát.

4.1.3. Siêu tham số (hyperparameters)

Trong thực nghiệm, ta cần lựa chọn một số siêu tham số quan trọng:

- **learning rate (lr)**: tốc độ học, ví dụ **0.001**.
- **batch size**: số mẫu trong mỗi batch cập nhật, ví dụ **32**.
- **epochs**: số vòng quét qua toàn bộ tập huấn luyện, ví dụ **10–20**.
- **dropout**: tỉ lệ dropout trong quá trình cập nhật, ví dụ **0.2** hoặc **0.5**.
- **cấu hình CNN** bên trong spaCy:
 - kiến trúc: **"simple_cnn"**
 - **exclusive_classes**: **True** (vì đây là phân loại đa lớp loại trừ lẫn nhau).

Các giá trị trên được chọn dựa trên kinh nghiệm và có thể tinh chỉnh thêm bằng cách theo dõi kết quả trên tập validation.

4.1.4. Cách định dạng dữ liệu theo spaCy – trường "cats"

Để huấn luyện với **TextCategorizer**, spaCy yêu cầu mỗi mẫu dữ liệu có dạng:

```
(text, {"cats": {"neg": 0/1, "neu": 0/1, "pos": 0/1}})
```

Trong đó:

- `text` là chuỗi review (string).
- `"cats"` là một dictionary chứa các nhãn và nhãn one-hot.

Ví dụ:

```
("This movie was amazing!", {"cats": {"neg": 0, "neu": 0, "pos": 1}})
("It was an average film.", {"cats": {"neg": 0, "neu": 1, "pos": 0}})
("Terrible plot and bad acting.", {"cats": {"neg": 1, "neu": 0, "pos": 0}})
```

Lưu ý:

- Vì là **multiclass**, mỗi mẫu chỉ có duy nhất một nhãn được gán giá trị 1.
- Trong code, ta sẽ chuyển cột `label` (0, 1, 2) trong DataFrame sang format `"cats"` như trên.

4.1.5. Quy trình huấn luyện với spaCy

Quy trình tổng quát:

1. Chuẩn bị dữ liệu:

- Có các DataFrame: `train_df`, `val_df`, `test_df`
- Mỗi DataFrame có ít nhất hai cột:
 - `review`: chuỗi văn bản
 - `label`: số nguyên 0 (neg), 1 (neu), 2 (pos)

2. Chuyển dữ liệu sang định dạng spaCy:

- Viết hàm `df_to_spacy_format(df)`
- Trả về list các phần tử `(text, {"cats": ...})`.

3. Tạo pipeline spaCy:

- Khởi tạo `nlp = spacy.blank("en")`.
- Thêm component `textcat_multiclass` vào pipeline.
- Đăng ký các nhãn `neg, neu, pos`.
- Gọi `nlp.initialize()` để khởi tạo tham số mô hình.

4. Huấn luyện theo epochs:

- Lặp qua `epoch = 1..n_epochs`.
- Shuffle dữ liệu train.
- Chia thành các mini-batch.
- Với mỗi batch:
 - Tạo danh sách `Example` từ `(text, annotations)`.
 - Gọi `nlp.update(examples, drop=dropout, losses=losses)` để cập nhật tham số qua gradient descent.
- In ra loss sau mỗi epoch.
- Có thể đánh giá nhanh accuracy/F1 trên tập validation.

5. Lưu mô hình:

- Gọi `nlp.to_disk("spacy_cnn_multiclass")` để lưu mô hình đã huấn luyện.
- Mô hình này sẽ được sử dụng lại ở phần Web App và phần đánh giá.

4.1.6. Code chi tiết (convert dữ liệu → train → lưu model)

Giả sử Cụ đã có:

- `train_df, val_df, test_df`
- Mỗi DataFrame có cột "`review`" (text) và "`label`" (0, 1, 2).

```
import spacy
from spacy.util import minibatch
from spacy.training import Example
import random
import numpy as np

# Ánh xạ nhãn số sang tên lớp
LABEL_MAP = {0: "neg", 1: "neu", 2: "pos"}

def df_to_spacy_format(df):
    """
    Chuyển DataFrame (review, label) sang list (text, {'cats': {...}})
    cho spaCy.
    """
    data = []
    for text, label in zip(df["review"], df["label"]):
        cats = {lbl: 0.0 for lbl in LABEL_MAP.values()}
        cats[LABEL_MAP[label]] = 1.0
        data.append((text, {"cats": cats}))
    return data

def evaluate_model(nlp, data):
    """
```

Đánh giá nhanh accuracy trên một tập dữ liệu spaCy-format.

```
data: list (text, {'cats': {...}})
"""

if len(data) == 0:
    return 0.0

correct = 0
total = 0

for text, ann in data:
    doc = nlp(text)
    # doc.cats là dict: {'neg': prob, 'neu': prob, 'pos': prob}
    pred_label = max(doc.cats, key=doc.cats.get)
    true_label = max(ann["cats"], key=ann["cats"].get)
    if pred_label == true_label:
        correct += 1
    total += 1

return correct / total
```

1. Chuyển train/val/test sang định dạng spaCy

```
train_data = df_to_spacy_format(train_df)
val_data    = df_to_spacy_format(val_df)
test_data   = df_to_spacy_format(test_df)
```

2. Khởi tạo pipeline spaCy rỗng

```
nlp = spacy.blank("en")
```

```
# 3. Thêm component textcat_multiclass
textcat = nlp.add_pipe(
    "textcat_multiclass",
    config={
        "exclusive_classes": True,    # các lớp loại trừ nhau
        (multiclass)
        "architecture": "simple_cnn" # kiến trúc CNN đơn giản của
spaCy
    }
)
```

```
# 4. Đăng ký nhãn cho bộ phân loại
for label_name in LABEL_MAP.values():
    textcat.add_label(label_name)
```

```
# 5. Khởi tạo tham số mô hình
nlp.initialize()
```

```
# 6. Thiết lập siêu tham số huấn luyện
n_epochs    = 10
batch_size  = 32
dropout     = 0.2
```

```
best_val_acc = 0.0
best_model_path = "spacy_cnn_multiclass"
```

```
for epoch in range(1, n_epochs + 1):
```



```
random.shuffle(train_data)

batches = minibatch(train_data, size=batch_size)

losses = {}

# Huấn luyện trên từng batch
for batch in batches:
    texts, annotations = zip(*batch)
    examples = []
    for text, ann in zip(texts, annotations):
        doc = nlp.make_doc(text)
        examples.append(Example.from_dict(doc, ann))

    nlp.update(
        examples,
        drop=dropout,
        losses=losses
    )

# Tính loss huấn luyện
train_loss = losses.get("textcat", 0.0)

# Đánh giá nhanh trên tập validation
val_acc = evaluate_model(nlp, val_data)

print(f"Epoch {epoch}/{n_epochs} - "
      f"Train loss: {train_loss:.4f} - "
      f"Val accuracy: {val_acc:.4f}")
```

```
# Lưu lại mô hình tốt nhất theo validation accuracy
if val_acc > best_val_acc:
    best_val_acc = val_acc
    nlp.to_disk(best_model_path)
    print(f"    -> Lưu mô hình tốt nhất với Val Acc =
{best_val_acc:.4f}")

print(f"Huấn luyện hoàn tất. Mô hình tốt nhất được lưu tại:
{best_model_path}")
```